Caruca v2

LLM-based specification mining for opaque shell commands.

Tiran's Research Objectives & Requirements

For Early Review with Drs. Greenberg and Eiers

Source	ai_docs/prep/master_idea.md
Generated	August 29, 2026 at 11:09 AM
Pages	8

Table of Contents

Master Idea Document

End Goal

With caruca_v2, I want to determine, through rigorous instrumented comparison, whether an LLM-based approach can match or exceed Caruca v1's performance, correctness, and coverage, not just assume it can. I'll do this by building a naive single-prompt LLM baseline first, measuring it three-way against v1 and ground truth on cost, correctness, and percent-change, with a secure sandboxed execution environment for testing arbitrary, including destructive, command permutations. An agentic Claude Code pipeline rebuild follows later as separate work.

This works toward two objectives: near-term, proving that an LLM-based approach can perform Caruca-style specification mining at all and demonstrating its efficacy; longer-term, producing the evidence and documentation needed to revise and resubmit the Caruca white paper (not accepted in its original submission), incorporating v2's findings and a clear account of what changed between v1 and v2 and why.

Specific Problem

Today's only option for Caruca-style specification mining is v1's hand-written, hard-to-extend pipeline (6,520 LOC; the LLM is used only for syntax-spec inference, and even that step is currently broken against modern DSPy). This leads to slow iteration and no existing measurement of what an LLM-first approach would cost or how well it would work.

The problem is sharper than just v1's maintainability: v1's pipeline is fundamentally scoped to commands with well-documented man pages and a testable, non-destructive execution model. It doesn't generalize to arbitrary executables: binaries with sparse or no documentation, tools whose behavior depends on binary format inspection rather than text parsing, or commands (like `rm`) that are destructive enough to require a secure sandbox before any invocation permutation can even be run. Before an LLM-based approach can be judged fairly against v1, the evaluation itself needs infrastructure v1 never had to build: a safe execution environment for destructive commands, and a plan for what auxiliary tooling an LLM would need to handle commands outside the man-page-documented, non-destructive sweet spot v1 was designed for.

Stakeholders & Consumers

Research Stakeholders

- Tiran Dagan — implementer
- Prof. Michael Greenberg (Stevens) — PI/advisor for Caruca; primary reviewer of Caruca-specific results
- Prof. William Eiers (Stevens) — PhD advisor; methodology sounding board, not Caruca-specific

White Paper Co-Authors

Prospective co-authors on the paper resubmission, and evaluators of the approach in the meantime:

- Evangelos Lamprou (Brown)
- Seong-Heon Jung (NYU)
- Mayank Keoliya (UPenn)
- Lukas Lazarek (Brown)
- Konstantinos Kallas (UCLA)
- Nikos Vasilakis (Brown)

Downstream Spec Consumers

Systems the output must satisfy, same as v1's, per the paper's §7:

- PaSh, POSH, ShellCheck, Shseer — each expects its own annotation shape

Operators

- Whoever runs the tooling day to day — currently just Tiran via CLI; revisit if a web GUI enters scope

Evaluation Criteria & Success Definition

1. **Correctness/fidelity** — how closely produced specifications/annotations match v1 and ground truth (reusing v1's §7.2 `cmp_specs.py` methodology)
2. **Cost/performance** — wall-clock, tokens, $ per command, captured for both v1's LLM step and the new system, so we can say whether v2 matches, exceeds, or trails v1 on cost/speed
3. **Coverage** — command behaviors, flags, and configurations handled, including how far v2 extends beyond v1's man-page-documented, non-destructive sweet spot
4. **Consistency/reproducibility** — variance across repeated runs (LLM nondeterminism is itself a reportable result)
5. **Percent-change roll-up** — an explicit v1-vs-v2 delta computed per dimension above (e.g. "+/-N% on Q2 exact-match," "Nx cost"), giving the comparison one clear headline number per dimension
6. **Reduction in hand-encoded logic** — how much procedural code is replaced for equal-or-better output on dimensions 1-4

No concrete target numbers yet; this is deliberately qualitative at this stage.

MVP Components (Flexible, provisional)

- **Baseline instrumentation** (extends v1)
- Add token/cost/wall-clock/model-ID telemetry to v1's existing LLM step, so it's a fair comparison point

Naive-LLM baseline (new, per Eiers' guidance)

- CLI tool: command binary + docs in, specification in a downstream-consumable format out
- Few-shot primed with concrete worked examples

Secure sandbox environment (new capability v2 adds beyond v1's model)

- Isolated execution for arbitrary command permutations, including destructive ones (e.g. `rm`)
- Must preserve strace/ptrace-level syscall visibility (hard constraint from the tracer)
- Candidate approaches (research done, not yet decided): extend v1's Docker/overlayfs backends for bulk CLI sweeps; Firecracker microVMs on the CloudLab host for stronger isolation on destructive commands (real guest kernel keeps `strace` working natively); gVisor as a possible middle ground pending a ptrace-compatibility spike

- One sandbox service, two front doors: the CLI calls it directly, the future web harness calls the same service over an API

LLM tool-augmentation layer (new capability v2 adds beyond v1's model)

- Starter set: binary/format inspection (e.g. binhex-style dumps), static/code analysis, help/usage-string extraction, for executables outside the man-page/text-documentation sweet spot
- Likely to grow (not yet committed): web search for documentation, GitHub connectivity for source/issue lookup, as specific target commands demand them

Evaluation harness

- Reuses v1's `eval/cmp_specs.py` methodology and `benchmarks/annotations/` ground truth
- Produces the three-way comparison (v1 vs. naive-LLM vs. ground truth) across the 6 evaluation dimensions
- Computes the percent-change roll-up (dimension 5) as an explicit output, not just raw numbers
- Output should be usable directly in a paper (tables/figures/methodology description), given the resubmission objective

Web GUI *(optional, not yet decided)*

- For running/visualizing comparisons interactively instead of via CLI/scripts
- Stretch idea: in-browser CPU emulation (v86-style) as a fully client-side "try it live" demo mode

[Add or adjust components as system design firms up]

Key Usage Scenarios

Researcher (Tiran)

1. **Run a baseline comparison** *As the researcher*, I want to run the naive-LLM baseline against a command and get back a spec plus token/cost/latency numbers, *so that* I can add a row to the comparison table without reconstructing measurements after the fact.

Diff against ground truth *As the researcher*, I want to diff the naive-LLM's spec against v1's ground truth using the existing `cmp_specs.py` methodology, *so that* correctness numbers are directly comparable to the paper's Q2 results.

Safely sweep destructive commands *As the researcher*, I want to run invocation permutations of destructive commands (e.g. `rm`) inside the secure sandbox, *so that* coverage testing isn't limited to the non-destructive commands v1's model could safely handle.

Track v1-to-v2 deltas as they happen *As the researcher*, I want every methodological deviation between v1 and v2 logged as it's made, *so that* the paper resubmission's "what changed and why" section can be written from records, not reconstructed from memory.

Advisor/Reviewer (Greenberg, Eiers, co-authors)

1. **See the three-way comparison** *As a reviewer*, I want the v1 / naive-LLM / ground-truth comparison summarized per dimension with the percent-change roll-up, *so that* I can judge how far the LLM-based approach extends what v1 could do, and whether it's strong enough to support a resubmission.

Author Team (Tiran + co-authors, for resubmission)

1. **Paper-ready output** *As the author team*, we want the evaluation harness's results in a form usable directly in a paper (tables/figures/methodology description), *so that* drafting the resubmission doesn't require re-deriving results from scratch.

System/Background

1. **Automatic telemetry capture** — When a baseline or naive-LLM run completes, then its telemetry (tokens, cost, wall-clock, model ID, seed) is recorded alongside the output, so no measurement has to be reconstructed later.
2. **Sandbox teardown** — When a sandboxed execution finishes (or crashes), then the isolated environment is torn down automatically, so no destructive side effect can leak to the host regardless of outcome.

Future / Stretch Directions *(not committed)*

- **Agentic full-pipeline rebuild** — replacing configuration generation, tracing orchestration, and specification derivation with Claude Code/Agent-SDK reasoning (Tiran's original proposal; deferred per Eiers until the naive baseline + evaluation harness exist)
- **Web GUI/dashboard** — for running and visualizing comparisons interactively, if the CLI-first approach turns out to be limiting
- **In-browser CPU emulation demo mode** — a fully client-side, zero-server-trust way to try a single command live in the web GUI (v86/TinyEMU-style), once the web GUI itself is in scope
- **Expanded tool-augmentation** — web search for documentation, GitHub connectivity for source/issue lookup, added as specific target commands demand them
- **Expanded command coverage** — beyond whatever initial command set the naive baseline targets first